# Project Proposal & Literature Review

Lightweight Learned Image Compression for Laptop-Class Training

[TODO: Student A] Mohamed Abdelazeem Zeyad Tarek

Milestone 1
CSE681 — Digital Image Processing

**Abstract**

This project investigates learned image compression under limited computational resources, with the goal of reproducing a compact hyperprior-based codec that can be trained within the project period on laptop-class hardware. The model will use reduced channel widths and latent dimensions, and its rate–distortion performance will be evaluated using peak signal-to-noise ratio (PSNR) and multi-scale structural similarity (MS-SSIM) as functions of bits per pixel (BPP). The evaluation will compare the learned codec with a conventional JPEG baseline and published ELIC operating points, while examining how reducing model width and latent channel count affects reconstruction quality and computational cost. Training will use random $256 \times 256$ crops from DIV2K, while final image-quality evaluation will use the Kodak dataset and, subject to availability and time, Tecnick.

## 1 Problem Statement and Motivation

Image compression aims to represent visual data using fewer bits while preserving the information and perceptual quality needed for reconstruction. Conventional codecs such as JPEG provide practical and widely used solutions, but learned image compression offers an alternative in which an encoder, a probability model, and a decoder are optimized jointly from data. In particular, hyperprior-based methods estimate the statistical properties of latent representations and use this information to support more efficient entropy coding (Ballé et al., 2018; **?**).

Despite their promising rate–distortion performance, learned codecs can require substantial computational resources and model capacity. Reproducing a large architecture may therefore be impractical within a short academic project or on laptop-class hardware. The central problem addressed in this work is whether a compact, reduced-width hyperprior codec can be trained and evaluated within the available time and compute budget, while retaining useful compression performance.

The project has four connected objectives. First, it will reproduce a compact hyperprior-based learned codec, reducing the channel width and latent channel count to make training more manageable. Second, it will measure rate–distortion performance using PSNR and MS-SSIM against BPP, and compare the results with a JPEG baseline and published ELIC operating points. Third, it will investigate the trade-off between reconstruction quality and model complexity as the network width and latent dimensionality are reduced. Finally, it will document practical considerations such as training feasibility, model configuration, and decoding cost.

The experimental design is intended to make the comparison meaningful rather than simply compare nominal quality settings. DIV2K training and validation images will provide the training source, with random $256 \times 256$ crops used during training. The Kodak dataset, containing 24 images, will serve as the primary evaluation benchmark; Tecnick may be added if time and dataset

availability permit. JPEG will provide a conventional baseline, while ELIC will provide a reference to a more recent efficient learned-compression approach. Comparisons will be based on measured BPP and image-quality metrics, with care taken to distinguish reproduced experiments from operating points reported in prior publications.

This work connects directly to digital image processing through transform-based representation, quantization, entropy coding, reconstruction, and objective image-quality assessment. Its motivation is not only to obtain a competitive rate–distortion curve, but also to understand the engineering trade-offs that arise when a learned compression architecture is scaled down for limited hardware. A successful outcome will establish a reproducible evaluation framework and clarify how much model capacity can be reduced before compression quality or practical usability is significantly affected.

## 2 Related Work

**[TODO: Student A/B: Summarize at least four references, including at least two from 2023–2026. Explain how the work informs this project. Keep the bibliography entries in the shared references.bib file.]**

Table 1: Key references and their relevance to this project.

| Reference | Year | Relevance |
|---|---|---|
| (Ballé et al., 2018) | 2018 | Scale-hyperprior learned image codec. |
| (He et al., 2022) | 2022 | Efficient learned image compression architecture. |
| **[TODO: Recent reference]** | **[TODO: Year]** | **[TODO: Relevance]** |
| **[TODO: Recent reference]** | **[TODO: Year]** | **[TODO: Relevance]** |

## 3 Dataset and Baseline Plan

The Kodak image dataset is used as the common evaluation dataset for the compression methods in this project. It contains 24 test images, each with a resolution of $768 \times 512$ pixels. The same original images will be used to evaluate both the conventional JPEG baseline and the learned compression model. Using an identical test set makes the comparison more consistent.

The JPEG baseline is implemented in Python using the Pillow library. JPEG quality settings are varied to obtain multiple operating points with different file sizes and reconstruction qualities. For every compressed image, the actual output file size is recorded. The compressed file is then decoded, and the reconstructed image is compared with its original Kodak image.

The current preliminary plots contain five averaged JPEG operating points. The exact quality settings associated with each point should be taken from the experiment CSV rather than inferred from the plotted coordinates. The evaluation script can also be run for the full planned quality list, if required by the team protocol.

The JPEG results establish a conventional compression baseline. Once the learned codec is available, it will be evaluated on the same Kodak images using the same metric definitions.

**Dataset source:** https://r0k.us/graphics/kodak/kodak/

# 4 Proposed Method

We plan to develop a reduced-width learned image-compression model based on the scale-hyperprior approach of Ballé et al. (2018). As shown in Figure 1, the encoder transforms an image into a latent representation. A hyperprior network produces additional information that helps estimate the latent distribution for entropy coding. The decoder uses the coded latents and hyperprior information to reconstruct the image.

To study the effects of model capacity and target bitrate, we will evaluate a small set of channel-width configurations and rate settings. For each selected configuration, the training loop will process a batch of RGB image crops. The encoder and hyperprior produce quantized latents, and the decoder reconstructs each crop. We will train the model by minimizing the rate–distortion objective in Eq. 1:

$$\mathcal{L} = R + \lambda D, \tag{1}$$

where $R$ is the estimated bitrate in bits per pixel, $D$ is the reconstruction distortion, and $\lambda$, hyperparameter which controls the trade-off between bitrate and image quality. We will use mean squared error (MSE), defined in Eq. 2, for the distortion term:

$$D = \frac{1}{3HW} \sum_{c=1}^{3} \sum_{u=1}^{H} \sum_{v=1}^{W} (x_{cuv} - \hat{x}_{cuv})^2 , \tag{2}$$

where $x$ is the input crop, $\hat{x}$ is its reconstruction, and $c$ indexes the three RGB channels. The optimizer will backpropagate this loss to update the model weights over repeated batches. We will record validation results and save model checkpoints during training. The exact widths, rate settings, and training parameters will be selected after a feasibility test. The likelihood-based rate is used for training; final reported BPP will be calculated from the actual encoded bitstream.

After training, each selected model will be evaluated on the same Kodak images as the JPEG baseline. We will encode and decode the images and compare BPP, PSNR, and MS-SSIM using the evaluation pipeline described below. Final BPP will use the actual encoded bitstream size.

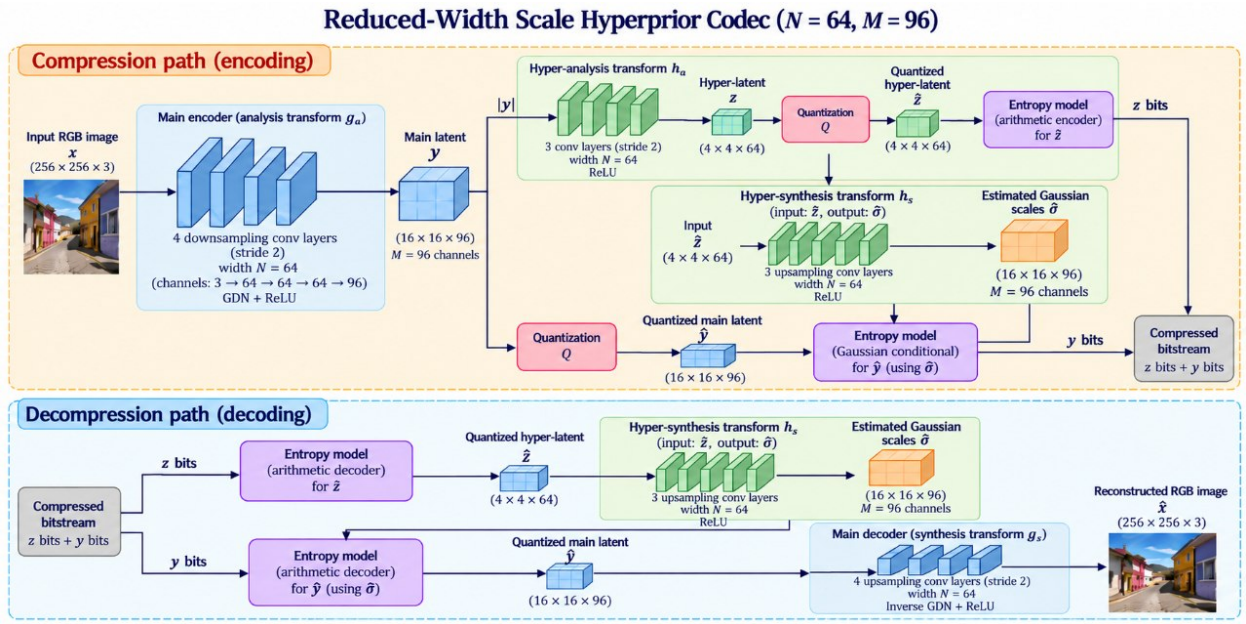



Figure 1: Planned architecture for the learned image-compression codec.

# 5 Evaluation Plan

The evaluation is designed to compare compression rate, reconstructed image quality, and decoding performance. Three primary image compression metrics are used.

- **Bits per pixel (BPP):** BPP measures the average number of encoded bits per image pixel. For a compressed file with size measured in bytes, BPP is calculated as shown in Eq. 3:

$$\text{BPP} = \frac{8 \times \text{compressed file size (bytes)}}{\text{image width} \times \text{image height}}. \tag{3}$$

  A lower BPP indicates a smaller encoded representation.

- **Peak signal-to-noise ratio (PSNR):** PSNR measures pixel-level reconstruction error and is reported in decibels (dB). For images represented with a maximum possible pixel value $MAX$, PSNR is calculated from the MSE as shown in Eq. 4:

$$\text{PSNR} = 10 \log_{10} \left( \frac{MAX^2}{\text{MSE}} \right). \tag{4}$$

  Higher PSNR generally indicates lower reconstruction error.

- **Multi-scale structural similarity (MS-SSIM):** MS-SSIM compares structural information at multiple image scales. Its commonly used range is 0 to 1, with values closer to 1 indicating greater structural similarity.

For each JPEG operating point, BPP, PSNR, and MS-SSIM are calculated for the Kodak images. The per-image measurements are retained, and the average metric values are used to produce the summary plots. BPP is calculated from the actual compressed file size, rather than from an estimated entropy or target bitrate.

Two rate–distortion plots are generated:

- Average PSNR (dB) versus BPP.
- Average MS-SSIM versus BPP.

The same evaluation procedure will be applied to the learned codec. The comparison will use the same original Kodak images, image dimensions, color handling, and metric definitions. The final report will include the individual measurements where appropriate, averages, and rate–distortion curves. Decoding time will also be measured on the same hardware environment for both methods. Timing will use warm-up runs and repeated measurements; file reading time should be excluded where practical so that the measurement focuses on decoding.

## 5.1 Preliminary JPEG Baseline Results

The first evaluation experiment establishes the JPEG baseline on the Kodak dataset. The current plots show five operating points, with average BPP values spanning approximately 0.33 to 2.35 bits per pixel. The values below are approximate readings from the plotted figures; the experiment CSV should be treated as the authoritative source for exact numerical results.

Figure 2 shows the average PSNR as a function of BPP. The plotted PSNR increases from approximately 26.7 dB at the lowest bitrate to approximately 38 dB at the highest bitrate. This

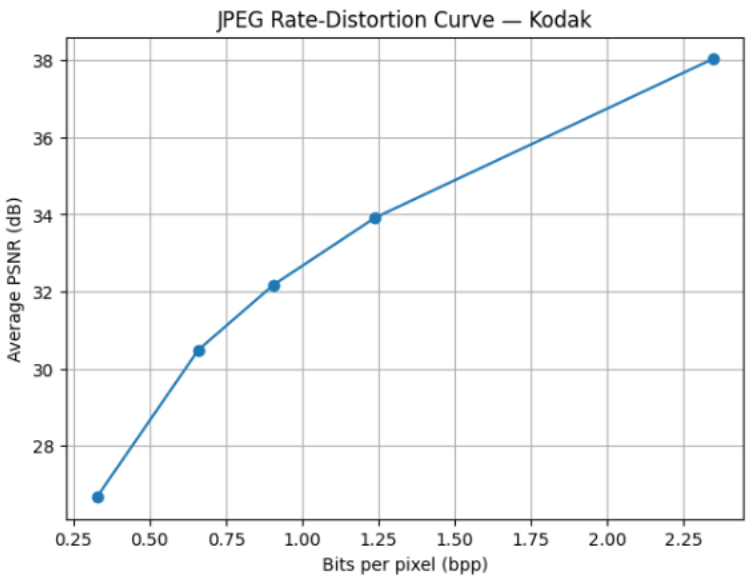



Figure 2: JPEG rate–distortion curve using average PSNR on the Kodak dataset.

illustrates the expected rate–distortion trade-off: allocating more bits generally improves reconstruction quality.

Figure 3 shows the average MS-SSIM at the same plotted operating points. The value increases from approximately 0.894 to approximately 0.993 as BPP increases. The curve also suggests that the improvement in MS-SSIM becomes smaller at higher bitrates, which is consistent with diminishing perceptual-quality gains as more bits are allocated.

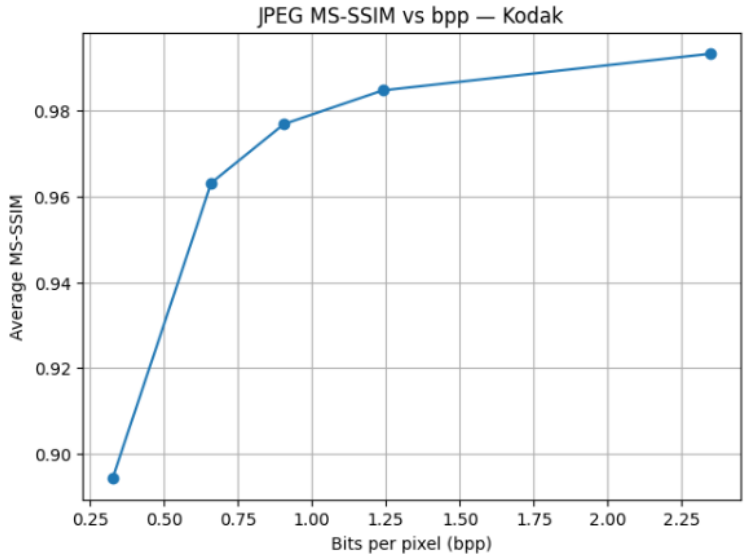



Figure 3: Average JPEG MS-SSIM versus BPP on the Kodak dataset.

These results are preliminary baseline measurements, not a comparison against the learned codec. The learned-codec results will be added after the trained model and its actual bitstream encoder/decoder are available.

### 5.2 Evaluation Implementation

The evaluation pipeline is implemented in Python. The main libraries used are:

- **Pillow (PIL):** JPEG compression and image decoding.
- **NumPy:** Image-array processing and numerical operations.
- **Pandas:** Organizing and exporting per-image and averaged experimental results to CSV.
- **Matplotlib:** Generating the rate–distortion plots.
- **scikit-image:** PSNR calculation.
- **pytorch-msssim:** MS-SSIM calculation.
- **PyTorch:** Tensor handling required by the MS-SSIM implementation.

The pipeline processes the Kodak images at each JPEG quality setting, saves the compressed outputs, records their file sizes, decodes them, and calculates the quality metrics against the corresponding original images. The measurements are saved in CSV format so that plots and summary tables can be regenerated.

The metric-calculation functions are kept separate from the codec implementation. This allows the same evaluation functions to be reused for the learned compression model and helps maintain a consistent comparison between JPEG and the learned codec.

## 6 Feasibility and Compute Plan

The model will be implemented using the CompressAI `bmshj2018-hyperprior` architecture, which corresponds to the scale-hyperprior method. In CompressAI 1.2.8, quality levels 1–5 use $N = 128$ hyperprior channels and $M = 192$ latent channels; levels 6–8 use $N = 192$ and $M = 320$. These are standard model configurations and quality presets. We will use the standard configuration as a reference and test a limited number of reduced-width configurations, with the number of experiments depending on available compute. For each configuration, the rate–distortion setting, including the distortion metric and $\lambda$, will be recorded. Reduced-width models will be trained for this project rather than treated as pretrained standard CompressAI models.

We will begin by trying to train the model on a laptop. If the required runtime or memory is not enough, we will use Google Colab when available. The available hardware may vary, so we will record the device, model configuration, image crop size, batch size, and training time for each experiment.

Before beginning a full training run, we will first check that one training step completes, then run a short pilot to measure average step time and peak memory use. We have not completed these tests yet, so we cannot estimate the full training time. We will use the pilot results to select a feasible model width, crop size, and batch size, and adjust them if needed. During training, we will monitor progress and save checkpoints for evaluation.

### 6.1 Minimum Viable Deliverable

**[TODO: As a team, state the smallest working experiment that produces a valid comparison with JPEG. At minimum, it should include a working learned encoder/decoder, actual encoded bitstream size, and evaluation on the same Kodak images.]**

### 6.2 Stretch Goal

**[TODO: As a team, state what will be attempted if the MVP is on schedule by Week 5–6, such as additional model-width settings, another evaluation dataset, or comparison with published operating points.]**

## 7 Risks and Mitigations

- **Student A/B – Training may exceed available compute.** The reduced-width model may still require more memory or training time than the available laptop or Colab session can provide. The team will begin with a small feasibility run and reduce the batch size, crop size, channel width, or number of training steps if necessary. The selected configuration and any resulting limitations will be documented.
- **Student B/C – The learned codec may not provide a complete bitstream pipeline on schedule.** A model that produces reconstructed tensors but does not yet support actual encoding and decoding cannot be compared fairly using true file-size BPP. The team will prioritize an end-to-end encoder/decoder and record the actual bitstream size. If this remains incomplete, the report will clearly label any estimated rate results and will not present them as measured bitstream BPP.
- **Student C – Inconsistent evaluation may lead to an unfair comparison.** Differences in image dimensions, color conversion, cropping, metric implementation, or data range can change PSNR and MS-SSIM results. To mitigate this risk, the same original Kodak images and consistent preprocessing and metric definitions will be used for JPEG and the learned codec. The evaluation scripts will retain per-image measurements as well as dataset averages, and the source CSV files will be preserved so that plots can be regenerated and checked.
- **Student C – Incorrect bitrate accounting.** Using an estimated entropy value instead of the actual compressed file size could misrepresent the achieved compression rate. For JPEG, BPP will be calculated from the saved JPEG file size. For the learned codec, the calculation will use the complete encoded bitstream, including the data required by the decoder. Any difference between estimated and actual rate will be stated.
- **Student C – Decode-time measurements may be noisy or hardware-dependent.** Runtime can vary with device load, warm-up state, and software configuration. Measurements will therefore be taken on a documented device, after warm-up, over repeated runs. File I/O will be excluded where practical, and the timing method and hardware will be reported. If a consistent timing environment cannot be maintained, decode-time results will be presented as indicative rather than as a definitive hardware comparison.

## 8 Work Breakdown

## 9 Student C Deliverables and Current Status

The following evaluation tasks have been completed for the initial JPEG baseline:

- Downloaded and prepared the 24-image Kodak evaluation dataset.
- Implemented BPP calculation using the actual compressed JPEG file size and image dimensions.

Table 2: Task allocation by milestone and team member.

| Milestone | Student A | Student B | Student C |
|---|---|---|---|
| M1 (Week 3) | **[TODO: Dataset preparation and JPEG baseline tasks]** | Review hyperprior design; determine the parameters for the model and run a one-step feasibility test. | Prepare Kodak evaluation dataset; implement BPP, PSNR, and MS-SSIM calculations; establish the JPEG baseline methodology and generate preliminary plots. |
| M2 (Week 6) | **[TODO: Training/data pipeline progress]** | Implement the training and encode/decode pipeline; train the model and verify that it produces a valid bitstream. | Complete and validate JPEG experiments; organize per-image and averaged results; generate and document preliminary rate–distortion curves. |
| M3 (Week 9) | **[TODO: Final data/training contribution]** | Finalize the trained model and bitstream encoder/decoder; document the model and training settings and support the final evaluation. | Evaluate the learned codec on Kodak; compare it against JPEG using rate–distortion curves; measure decoding time; prepare final result tables and qualitative comparisons. |

- Implemented PSNR and MS-SSIM calculation functions.
- Ran JPEG compression experiments and collected preliminary measurements.
- Generated average PSNR–BPP and MS-SSIM–BPP plots.

The remaining evaluation tasks are to run the same pipeline on the learned codec, verify that BPP is based on the actual encoded bitstream, measure decoding time under a consistent protocol, and produce the final JPEG-versus-learned-codec comparison.

## References


Johannes Ballé, David Minnen, Saurabh Singh, Sung Jin Hwang, and Nick Johnston. Variational image compression with a scale hyperprior. In *International Conference on Learning Representations*, 2018.

Dailan He, Ziming Yang, Weiming Peng, Rui Wang, Hongwei Ma, Hong Cheng, et al. ELIC: Efficient learned image compression with unevenly grouped space-channel contextual adaptive coding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.